VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

WEEK-9

Simulate the following Disk scheduling algorithms. a) FCFS b) SSTF c) SCAN

AIM- To implement disk scheduling algorithms such as FCFS, SSTF, SCAN

ALGORITHM-

FCFS – First Come First Serve

1. Start
2. Read the number of disk requests.
3. Read the request queue.
4. Read the initial head position.
5. Set total Movement = 0.
6. Take the requests in the given order.
7. Calculate the head movement:
8. $| CurrentHead - Request |$
9. Add the movement to total Movement.
10. Make the current request the new head position.
11. Repeat until all requests are serviced.
12. Display the seek sequence and total head movement.
13. Stop

PROGRAM :

```
#include <stdio.h>
#include <stdlib.h>
int main() {
int queue[100], n, head, i, seek_count = 0;
printf("Enter the number of disk requests: ");
scanf("%d", &n);
printf("Enter the sequence of disk track requests:\n");
for (i = 0; i < n; i++) {
scanf("%d", &queue[i]);
}
printf("Enter the initial position of the disk head: ");
scanf("%d", &head);
// Calculate total seek operations
for (i = 0; i < n; i++) {
seek_count += abs(queue[i] - head);
head = queue[i];
}
printf("\nTotal number of seek operations = %d\n", seek_count);
return 0;
}
```

VNR VJIET

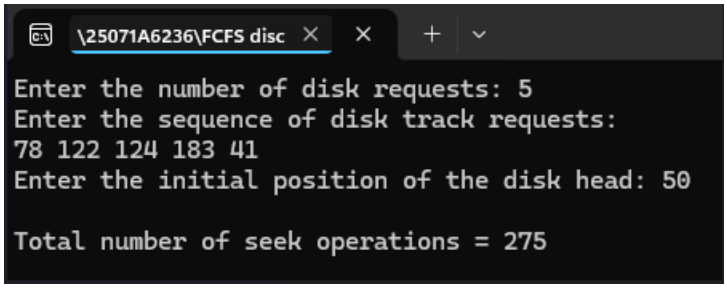
Name of the Laboratory:

Name of the Experiment:_____

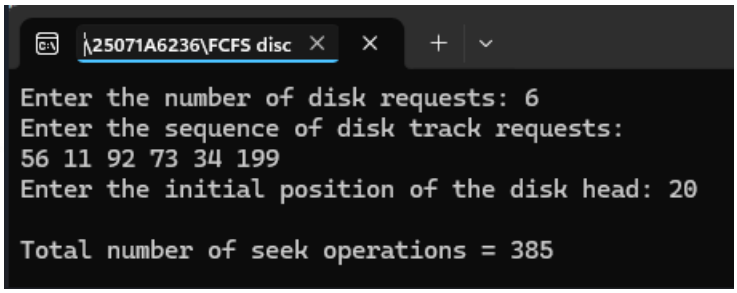
Experiment No:_____ **Date:**_____

OUTPUT :

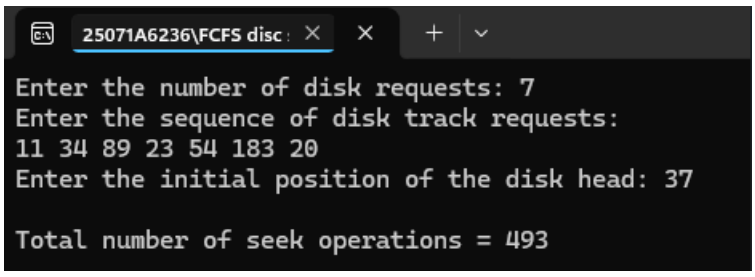
TEST CASE 1:



TEST CASE 2:



TEST CASE 3:



ALGORITHM:

2. SSTF – Shortest Seek Time First

1. Start
2. Read the number of disk requests.
3. Read the request queue.
4. Read the initial head position.
5. Mark all requests as unvisited.
6. Find the unvisited request having the minimum distance from the current head.
7. Calculate:
8. $Distance = | CurrentHead - Request |$
9. Service that requests.
10. Mark the request as visited.
11. Add the distance to totalMovement.
12. Make that request the new head position.
13. Repeat until all requests are serviced.
14. Display the seek sequence and total head movement.
15. Stop

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

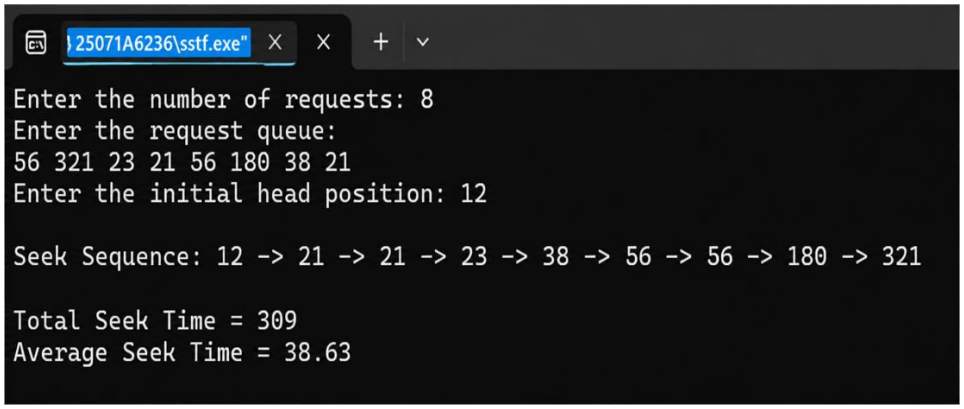
PROGRAM :

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
int n, head;
int request[50], visited[50];
int i, count, index, distance;
int minDistance;
int totalMovement = 0;
printf("=======================================\n");
printf("       DISK SCHEDULING - SSTF\n");
printf("=======================================\n");
printf("Enter number of requests: ");
scanf("%d", &n);
printf("Enter request queue:\n");
for(i = 0; i < n; i++)
{
scanf("%d", &request[i]);
visited[i] = 0;
}
printf("Enter initial head position: ");
scanf("%d", &head);
printf("\nSeek Sequence: %d", head);
for(count = 0; count < n; count++)
{
minDistance = 99999;
index = -1;
/* Find nearest unvisited request */
for(i = 0; i < n; i++)
```

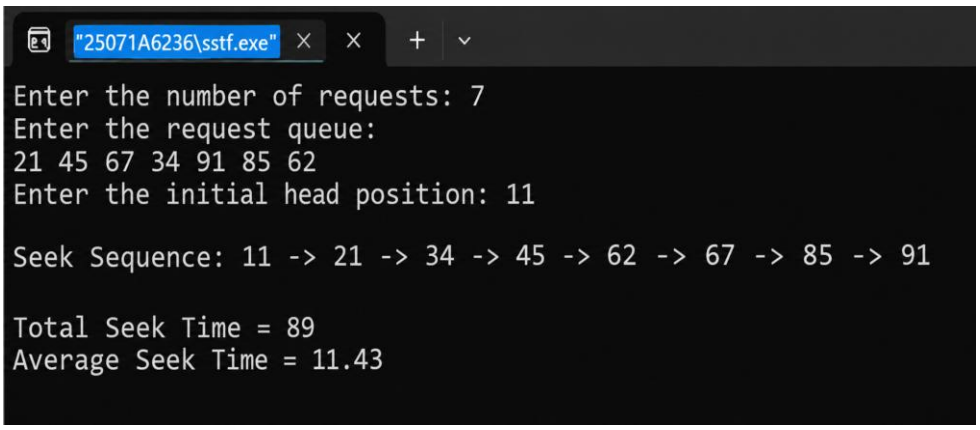
```
{
                if(visited[i] == 0)
                {
                distance = abs(head - request[i]);
        if(distance < minDistance)
        {
                minDistance = distance;
                index = i;
        }
        }
}
/* Service the nearest request */
visited[index] = 1;
totalMovement += minDistance;
head = request[index];
printf(" -> %d", head);
}
printf("\n\nTotal Head Movement = %d cylinders\n", totalMovement);
return 0;
}
```

OUTPUT :

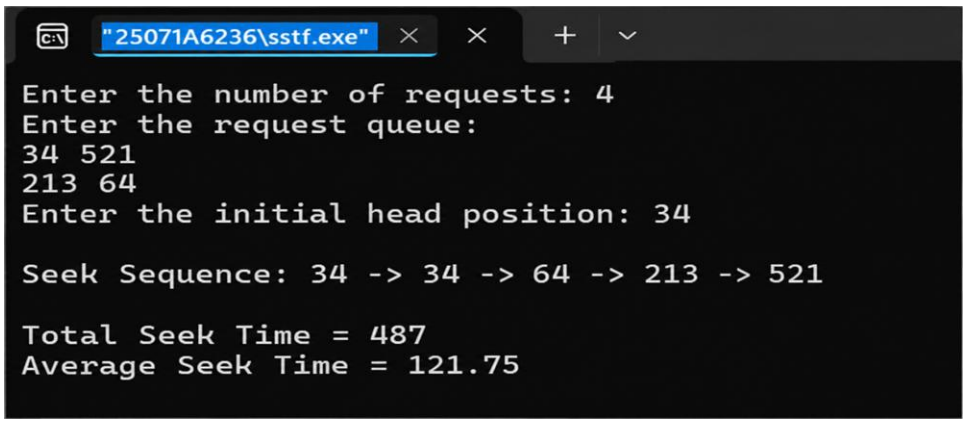
TEST CASE 1 :



TEST CASE 2 :



TEST CASE 3 :



3.**SCAN**

ALGORITHM :

1. Start
2. Read the number of disk requests.
3. Read the request queue.
4. Read the initial head position.
5. Read the disk size.
6. Read the direction of head movement.
7. Sort the request queue in ascending order.
8. If the direction is right:
    i. Service requests greater than the head in ascending order.
    ii. Move the head to the end of the disk.
    iii. Reverse the direction.
    iv. Service the remaining requests in descending order.
9. If the direction is left:
    i. Service requests smaller than the head in descending order.
    ii. Move the head to 0.
    iii. Reverse the direction.
    iv. Service the remaining requests in ascending order.
10. Calculate the total head movement.
11. Display the seek sequence and total head movement.
12. Stop

PROGRAM :

```
#include <stdio.h>

#include <stdlib.h>

// Comparison function for qsort

int compare(const void *a, const void *b) {

return (*(int*)a - *(int*)b);

}

int main() {

int queue[100], n, head, disk_size, i, j, seek_count = 0;

int left[100], right[100];

int l_size = 0, r_size = 0;

char direction;

printf("Enter the number of disk requests: ");

scanf("%d", &n);

printf("Enter the sequence of disk track requests:\n");

for (i = 0; i < n; i++) {

scanf("%d", &queue[i]);

}

printf("Enter the initial position of the disk head: ");

scanf("%d", &head);

printf("Enter the total disk size (e.g., 200): ");

scanf("%d", &disk_size);

printf("Enter the head movement direction ('H' for High, 'L' for Low): ");

scanf(" %c", &direction);
```

```
// Add boundaries to the scan path depending on the direction
if (direction == 'H' || direction == 'h') {
right[r_size++] = disk_size - 1; // Goes to the very end
} else {
left[l_size++] = 0; // Goes to the very beginning
}
// Separate requests into left and right sides of the head
for (i = 0; i < n; i++) {
if (queue[i] < head) {
        left[l_size++] = queue[i];
} else {
        right[r_size++] = queue[i];
}
}
// Sort both arrays to service them in order
qsort(left, l_size, sizeof(int), compare);
qsort(right, r_size, sizeof(int), compare);
printf("\nDisk Head Movement Sequence:\n%d", head);
// Process based on chosen direction
if (direction == 'H' || direction == 'h') {
// Service requests to the right (higher tracks)
```

```
for (i = 0; i < r_size; i++) {
        seek_count += abs(right[i] - head);
        head = right[i];
        printf(" -> %d", head);
}
// Reverse direction and service remaining requests on the left
for (i = l_size - 1; i >= 0; i--) {
        seek_count += abs(left[i] - head);
        head = left[i];
        printf(" -> %d", head);  }
} else {
// Service requests to the left (lower tracks)
for (i = l_size - 1; i >= 0; i--) {
        seek_count += abs(left[i] - head);
        head = left[i];
        printf(" -> %d", head);  }
// Reverse direction and service remaining requests on the right
for (i = 0; i < r_size; i++) {
        seek_count += abs(right[i] - head);
        head = right[i];
        printf(" -> %d", head); } }
printf("\n\nTotal number of seek operations = %d\n", seek_count);
return 0;   }
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

OUTPUT :

TEST CASE 1 :

```
25071A6236\SCAN disc
Enter the number of disk requests: 5
Enter the sequence of disk track requests:
78 122 124 183 41
Enter the initial position of the disk head: 50
Enter the total disk size (e.g., 200): 200
Enter the head movement direction ('H' for High, 'L' for Low): 199,0

Disk Head Movement Sequence:
50 -> 41 -> 0 -> 78 -> 122 -> 124 -> 183

Total number of seek operations = 233
```

TEST CASE 2 :

```
25071A6236\SCAN dis
Enter the number of disk requests: 6
Enter the sequence of disk track requests:
56 11 92 73 34 199
Enter the initial position of the disk head: 20
Enter the total disk size (e.g., 200): 200
Enter the head movement direction ('H' for High, 'L' for Low): 199,0

Disk Head Movement Sequence:
20 -> 11 -> 0 -> 34 -> 56 -> 73 -> 92 -> 199

Total number of seek operations = 219
```

TEST CASE 3 :

```
25071A6236\SCAN disc
Enter the number of disk requests: 7
Enter the sequence of disk track requests:
11 34 89 23 54 183 20
Enter the initial position of the disk head: 37
Enter the total disk size (e.g., 200): 200
Enter the head movement direction ('H' for High, 'L' for Low): 199,0

Disk Head Movement Sequence:
37 -> 34 -> 23 -> 20 -> 11 -> 0 -> 54 -> 89 -> 183

Total number of seek operations = 220
```

4. C SCAN

Algorithm for C-SCAN Disk Scheduling

1:Start.

2: Read the number of disk requests n.

3: Read all the disk request positions into an array.

4: Read the initial head position.

5: Sort the request positions in ascending order.

6: Find the position of the first request greater than or equal to the head position.

7: Move the disk head toward the higher-numbered cylinders, servicing all requests in that direction.

8: Continue moving until the end of the disk is reached.

9: After reaching the end, move the head directly to the beginning of the disk without servicing requests while returning.

10: From the beginning, continue moving toward the higher-numbered cylinders and service the remaining requests.

11: Calculate the total head movement.

12: Display the order of serviced requests and total head movement.

13: Stop.

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

PROGRAM :

```
#include <stdio.h>
#include <stdlib.h>
int main() {
int n, head, disk_size;
int requests[100];
int left[100], right[100];
int l = 0, r = 0;
int i, j, temp;
int total_movement = 0;
printf("Enter number of disk requests: ");
scanf("%d", &n);
printf("Enter disk size: ");
scanf("%d", &disk_size);
printf("Enter disk requests:\n");
for (i = 0; i < n; i++) {
scanf("%d", &requests[i]);  }
printf("Enter initial head position: ");
scanf("%d", &head);
/* Separate requests into left and right of head */
for (i = 0; i < n; i++) {
if (requests[i] < head)
        left[l++] = requests[i];
else
    right[r++] = requests[i];   }
/* Add disk ends */
right[r++] = disk_size - 1;
        left[l++] = 0;
        /* Sort left array */
        for (i = 0; i < l - 1; i++) {
        for (j = i + 1; j < l; j++) {
```

```
                if (left[i] > left[j]) {
                temp = left[i];
                left[i] = left[j];
                left[j] = temp; }  }  }
/* Sort right array */
for (i = 0; i < r - 1; i++) {
for (j = i + 1; j < r; j++) {
        if (right[i] > right[j]) {
        temp = right[i];
        right[i] = right[j];
        right[j] = temp ; }
         }
         }
printf("\nC-SCAN Disk Scheduling Order:\n");
printf("%d", head);
/* Move towards right */
for (i = 0; i < r; i++) {
total_movement += abs(head - right[i]);
head = right[i];
printf(" -> %d", head);
}
/* Jump from end to beginning */
total_movement += disk_size - 1;
head = 0;
printf(" -> %d", head);
/* Move towards right again */
for (i = 0; i < l; i++) {
if (left[i] == 0)
        continue;
total_movement += abs(head - left[i]);
printf(" -> %d", head); }
printf("\n\nTotal Head Movement = %d cylinders\n", total_movement);
return 0;
}
```

VNR VJIET

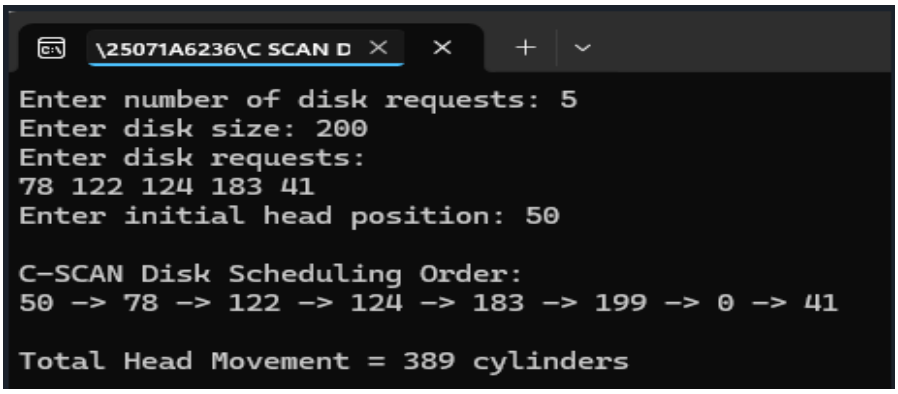
Name of the Laboratory:

Name of the Experiment:_____

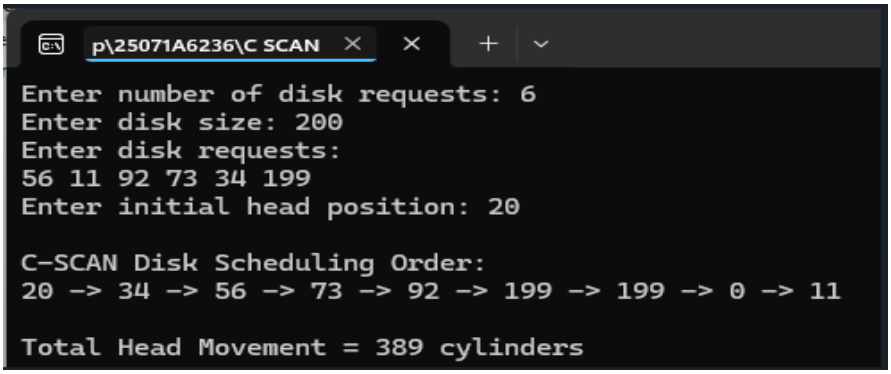
Experiment No:_____ **Date:**_____

OUTPUT :

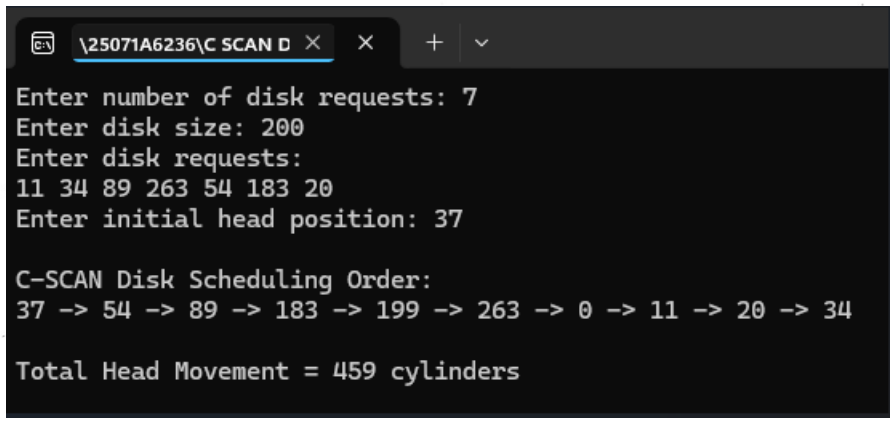
TEST CASE 1 :



TEST CASE 2:



TEST CASE 3 :



VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

*/

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

WEEK-10

Simulate the following Disk scheduling algorithms a) LOOK b) C-LOOK AIM-

To implement disk scheduling algorithms such as LOOK , C-LOOK

ALGORITHM-

LOOK

1. **Start**
2. Read the number of disk requests.
3. Read the request queue.
4. Read the initial head position.
5. Read the direction of head movement.
6. Sort the request queue in ascending order.
7. If the direction is **right**:
    - Service requests greater than the head in ascending order.
    - Stop at the **last request**.
    - Reverse direction.
    - Service remaining requests in descending order.
8. If the direction is **left**:
    - Service requests smaller than the head in descending order.
    - Stop at the **first request**.
    - Reverse direction.
    - Service remaining requests in ascending order.
9. Calculate total head movement.
10. Display the seek sequence and total head movement.
11. **Stop**

Program:

LOOK:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, head, direction;
    int request[50];
    int i, j, temp;
    int totalMovement = 0;

    printf("========================================\n");
    printf("       DISK SCHEDULING - LOOK\n");
    printf("========================================\n");

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &request[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter direction (1 = Right, 0 = Left): ");
    scanf("%d", &direction);
```

```
for(i = 0; i < n - 1; i++)
{
   for(j = i + 1; j < n; j++)
   {
      if(request[i] > request[j])
      {
         temp = request[i];
         request[i] = request[j];
         request[j] = temp;
      }
   }
}

printf("\nSeek Sequence: %d", head);

if(direction == 1)
{
   /* Move right */
   for(i = 0; i < n; i++)
   {
      if(request[i] >= head)
      {
         totalMovement += abs(head - request[i]);
         head = request[i];

         printf(" -> %d", head);
      }
```

```
    }

    /* Reverse direction */
    for(i = n - 1; i >= 0; i--)
    {
        if(request[i] < head)
        {
            totalMovement += abs(head - request[i]);
            head = request[i];

            printf(" -> %d", head);
        }
    }
}
else
{
    /* Move left */
    for(i = n - 1; i >= 0; i--)
    {
        if(request[i] <= head)
        {
            totalMovement += abs(head - request[i]);
            head = request[i];

            printf(" -> %d", head);
        }
    }

    /* Reverse direction */
```

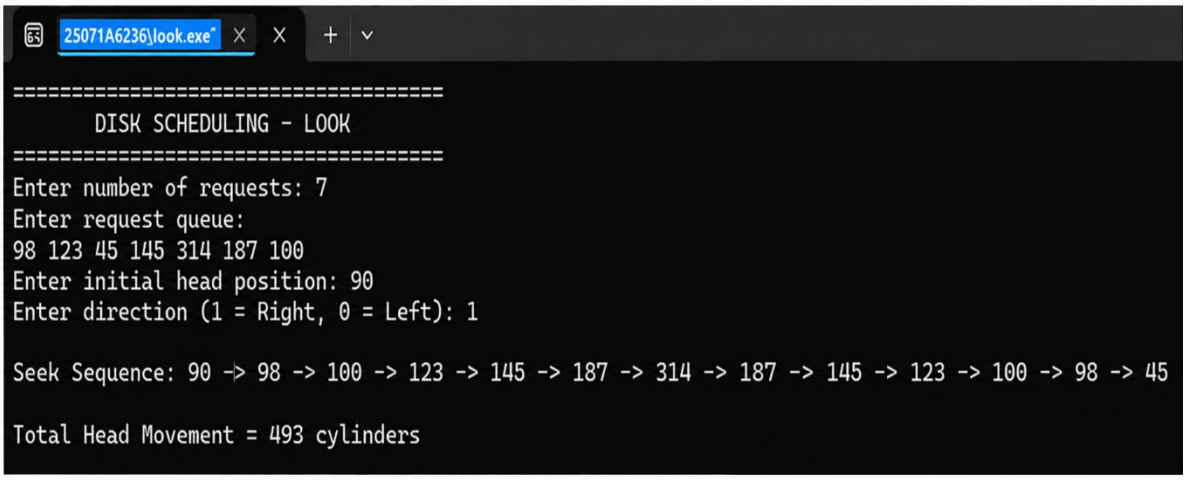
```
for(i = 0; i < n; i++)
        {
           if(request[i] > head)
           {
              totalMovement += abs(head - request[i]);
              head = request[i];

              printf(" -> %d", head);
           }
        }
    }
    printf("\n\nTotal Head Movement = %d cylinders\n", totalMovement);

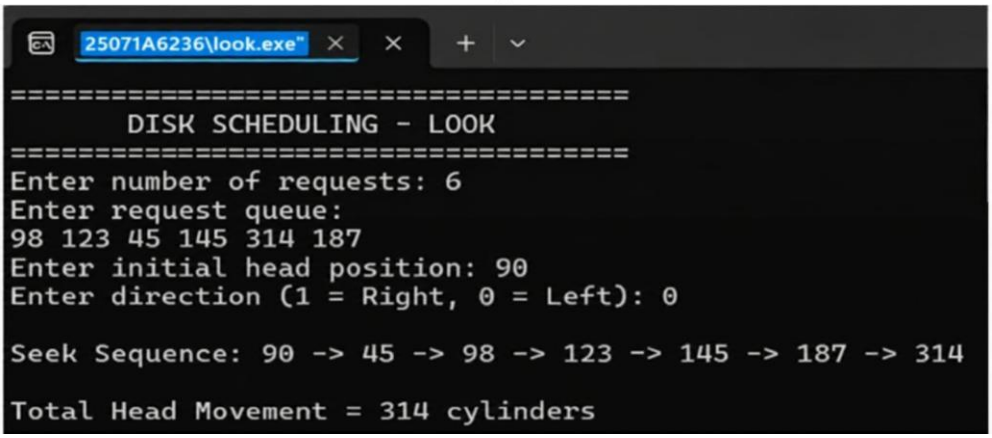
    return 0;
}
```

OUTPUT:

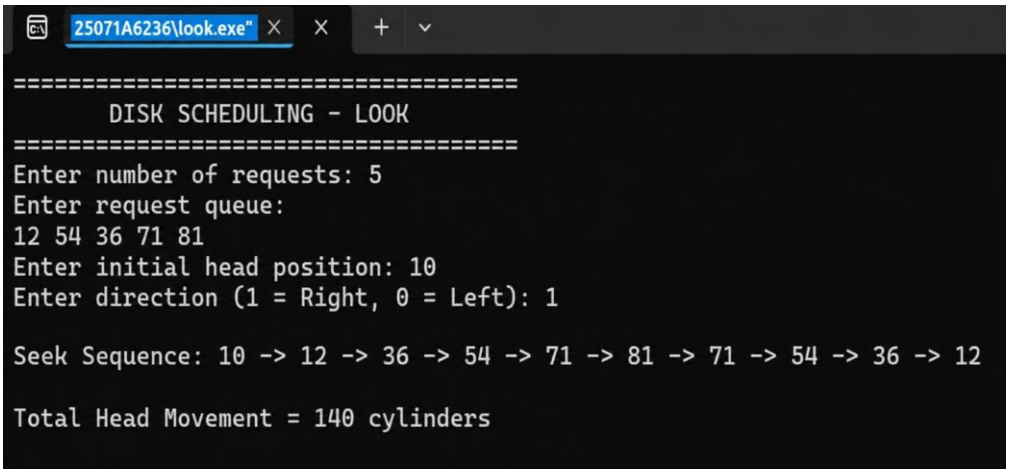
TESTCASE 1:



TESTCASE 2:



TEST CASE 3 :



b) C LOOK

ALGORITHM :

Start.

1. Read the number of disk requests n.
2. Read all the disk request positions.
3. Read the initial head position.
4. Read the direction of head movement:
5. 1 → Right
6. 0 → Left
7. Sort all the requests in ascending order.
8. Find the position of the head in the sorted request list.
9. If the direction is Right:
10. Move the head toward higher-numbered cylinders.
11. Service all requests greater than or equal to the head.
12. After reaching the highest request, jump directly to the lowest request.
13. Continue servicing the remaining requests in the same direction.
14. If the direction is Left:
15. Move the head toward lower-numbered cylinders.
16. Service all requests less than or equal to the head.
17. After reaching the lowest request, jump directly to the highest request.
18. Continue servicing the remaining requests in the same direction.
19. Calculate the total head movement.
20. Display the seek sequence and total head movement.

Stop.

PROGRAM:

C-LOOK:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, head, direction;
    int request[50];
    int i, j, temp;
    int totalMovement = 0;

    printf("========================================\n");
    printf("      DISK SCHEDULING - C-LOOK\n");
    printf("========================================\n");

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &request[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter direction (1 = Right, 0 = Left): ");
    scanf("%d", &direction);
    /* Sort requests */
    for(i = 0; i < n - 1; i++)
```

```
{
   for(j = i + 1; j < n; j++)
   {
      if(request[i] > request[j])
      {
         temp = request[i];
         request[i] = request[j];
         request[j] = temp;
      }
   }
}

printf("\nSeek Sequence: %d", head);
if(direction == 1)
{
   /* Move right */
   for(i = 0; i < n; i++)
   {
      if(request[i] >= head)
      {
         totalMovement += abs(head - request[i]);
         head = request[i];
  printf(" -> %d", head);
      }
   }
   for(i = 0; i < n; i++)
   {
      if(request[i] < head)
      {
```

```
printf(" -> %d", head);
            }
        }
        for(i = 0; i < n; i++)
        {
            if(request[i] < head)
            {
                totalMovement += abs(head - request[i]);
                head = request[i];

                printf(" -> %d", head);
            }
        }
    }
    else
    {
        for(i = n - 1; i >= 0; i--)
        {
            if(request[i] <= head)
            {
                totalMovement += abs(head - request[i]);
                head = request[i];

                printf(" -> %d", head);
            }
        }
        for(i = n - 1; i >= 0; i--)
        {
            if(request[i] > head)
            {
```

```
            totalMovement += abs(head - request[i]);
            head = request[i];

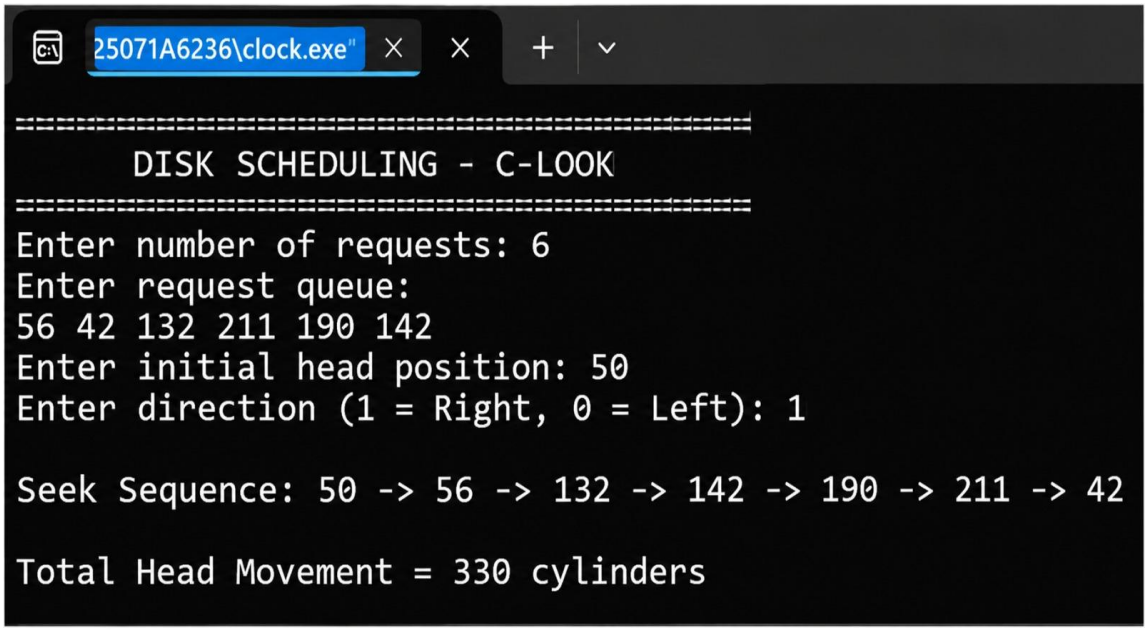
            printf(" -> %d", head);
        }
    }
}

printf("\n\nTotal Head Movement = %d cylinders\n", totalMovement);

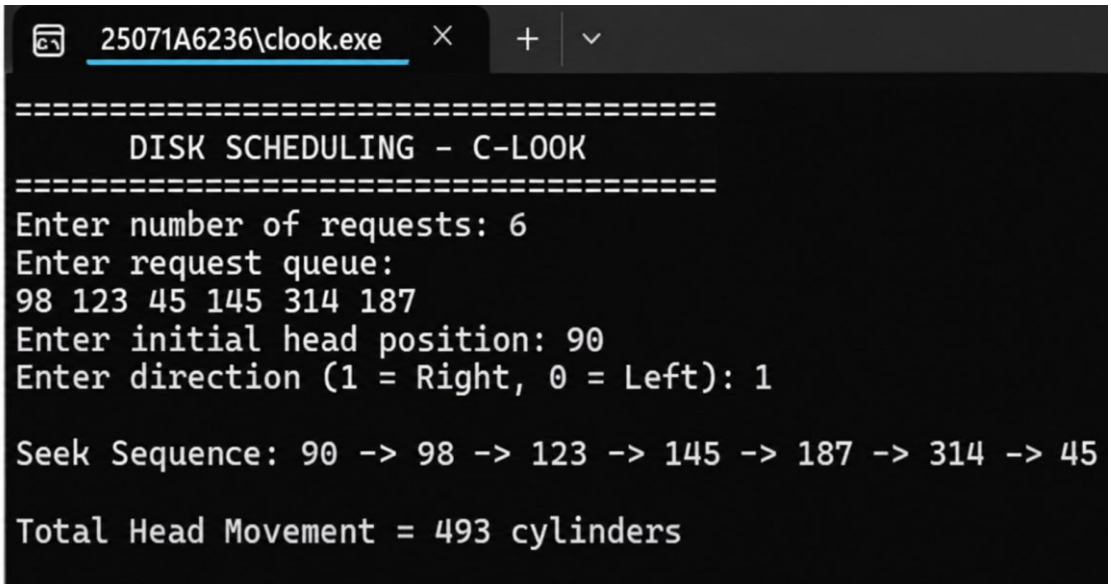
return 0;
}
```

OUTPUT:

TESTCASE 1:



TESTCASE 2:



TESTCASE 3:

